

NIRA — Personal AI Assistant Blueprint

NIRA is a personalized AI desktop assistant designed to act as a proactive coding and productivity companion. The project combines conversational AI, memory systems, multilingual voice interaction, safe tool execution, browser automation, and adaptive personality systems into a single modular architecture. The goal is to create an assistant that feels context-aware, remembers habits, adapts to workflows, and assists naturally during coding and daily tasks.

1. Core Project Goals

- Provide conversational assistance for coding, browsing, and task execution.
- Support multilingual conversations including Hindi, English, and Hinglish.
- Maintain persistent memory and personality consistency.
- Safely create, edit, search, and organize files and folders.
- Provide proactive suggestions based on user habits and workflows.
- Support voice interaction using wake-word activation and speech pipelines.
- Allow future model replacement without changing personality behavior.

2. Recommended System Architecture

The architecture should follow a modular layered design. The frontend layer manages the graphical interface, avatar system, and voice interaction. The core brain layer handles conversation orchestration, personality management, memory retrieval, planning, and context building. A separate tools layer manages file operations, browser automation, and coding tools. The security layer validates permissions and sandbox restrictions before execution.

```
NIRA/  
■■■ core/  
■■■ tools/  
■■■ security/  
■■■ memory/  
■■■ voice/  
■■■ ui/  
■■■ models/  
■■■ main.py
```

3. Recommended Tech Stack

Component	Recommended Technology
Main Backend	Python
UI Framework	PyQt6
Browser Automation	Playwright
Vector Memory	ChromaDB
Embeddings	BGE-small-en-v1.5
STT	Faster-Whisper
TTS	Piper TTS

Wake Word	OpenWakeWord
LLM API	OpenRouter
Local Runtime	Ollama
Agent Orchestration	LangGraph

4. Recommended Models

The recommended architecture uses a hybrid model approach to balance speed, hardware limitations, and quality. The primary LLM should run through API providers while speech systems and embeddings remain local.

Purpose	Model
Primary LLM	DeepSeek V3 via OpenRouter
Fallback LLM	Qwen 32B
Offline Backup	Phi-3 Mini
Speech-to-Text	Faster-Whisper
Text-to-Speech	Piper TTS
Embeddings	BGE-small-en-v1.5

5. Personality and Memory System

NIRA's personality should not depend entirely on prompts. A dedicated personality layer must exist independently from the LLM provider. The system should maintain user preferences, coding habits, workflow patterns, interests, and language styles. A dynamic context builder should inject these memories before every LLM request.

```
{
  "coding_habits": {
    "likes_music_while_coding": true
  },
  "preferred_languages": ["english", "hinglish"],
  "interests": ["AI", "anime", "web development"]
}
```

6. Voice Interaction Pipeline

Voice interaction should use a low-latency event-driven architecture. The wake-word detector continuously listens in lightweight mode. Once triggered, speech recording begins and stops automatically using voice activity detection.

```
Microphone
↓
Wake Word Detection
↓
Speech Recording
↓
Voice Activity Detection
↓
Speech-to-Text
↓
LLM Processing
↓
Text-to-Speech
```



Audio Playback

7. Security and Guardrails

Security is one of the most important aspects of the system because NIRA will receive authority to access files, execute operations, and automate tasks. The architecture must use strict permission validation and tool isolation.

- Restrict accessible directories using allowlists.
- Prevent deletion of critical system folders.
- Require confirmation for overwrite operations.
- Disable privilege escalation.
- Separate planning logic from execution logic.
- Implement risk classification for commands.

8. Important Implementation Techniques

The assistant should use interchangeable provider interfaces for STT, TTS, memory, and LLM systems. This makes future upgrades easier without changing NIRA's personality or architecture. Caching, async execution, and streaming responses should be used to reduce latency.

```
class LLMProvider:
    def generate(self, prompt):
        pass

class STTProvider:
    def transcribe(self, audio):
        pass
```

9. Recommended Python Libraries

Core Libraries: PyQt6, Playwright, LangGraph, ChromaDB, SentenceTransformers, Faster-Whisper, OpenWakeWord, Piper-TTS, SoundDevice, Torch, Transformers, Watchdog, Rich, PyAutoGUI, Python-dotenv, Tiktoken, FAISS-CPU.

10. Things to Keep in Mind

Do not overengineer the first version. Build a stable MVP first before attempting autonomous multi-agent behavior. Maintain modularity from the beginning. Avoid tightly coupling memory, voice systems, and model providers. Focus heavily on responsiveness and user experience. Small delays can break immersion in conversational assistants. The assistant should always feel safe, context-aware, and predictable.

11. Suggested Development Phases

Phase	Objective
Phase 1	Chat UI, memory, STT, TTS, basic tools
Phase 2	Habit tracking and semantic memory
Phase 3	Browser automation and workflows
Phase 4	Animated avatar and desktop overlay

NIRA has the potential to become a highly capable personalized AI companion if developed incrementally with a strong focus on architecture, safety, memory systems, and responsiveness. The modular approach recommended in this blueprint ensures scalability while allowing future upgrades to models and infrastructure without redesigning the assistant.